

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/220609357>

Design guidelines for software processes knowledge repository development

Article in *Information and Software Technology* · August 2011

DOI: 10.1016/j.infsof.2011.03.002 · Source: DBLP

CITATIONS

35

READS

116

4 authors:



Javier García Guzmán

University Carlos III de Madrid

78 PUBLICATIONS 323 CITATIONS

[SEE PROFILE](#)



Antonio de Amescua

University Carlos III de Madrid

114 PUBLICATIONS 546 CITATIONS

[SEE PROFILE](#)



Maria-Isabel Sanchez-Segura

University Carlos III de Madrid

56 PUBLICATIONS 521 CITATIONS

[SEE PROFILE](#)



Leonardo Bermón-Angarita

National University of Colombia

10 PUBLICATIONS 35 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Living Labs for User driven design and validation of transport services based on autonomous vehicles [View project](#)



Digital Education [View project](#)

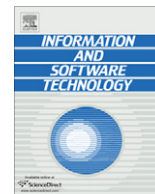
All content following this page was uploaded by [Leonardo Bermón-Angarita](#) on 21 July 2016.

The user has requested enhancement of the downloaded file.



Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Design guidelines for software processes knowledge repository development

Javier García^a, Antonio Amescua^a, María-Isabel Sánchez^a, Leonardo Bermón^{b,*}^a Software Engineering Lab, Department of Computer Science, University Carlos III of Madrid, Av. Universidad 30, 28911, Leganés, Spain^b Department of Computer Science, National University of Colombia, Campus La Nubia, Manizales, Colombia

ARTICLE INFO

Article history:

Received 26 July 2010

Received in revised form 15 March 2011

Accepted 16 March 2011

Available online 22 March 2011

Keywords:

Software engineering

Software process technology

Knowledge management

Agile development

Web 2.0

Wiki

ABSTRACT

Context: Staff turnover in organizations is an important issue that should be taken into account mainly for two reasons:**Context:**

1. Employees carry an organization's knowledge in their heads and take it with them wherever they go
2. Knowledge accessibility is limited to the amount of knowledge employees want to share

Objective: The aim of this work is to provide a set of guidelines to develop knowledge-based Process Asset Libraries (PAL) to store software engineering best practices, implemented as a wiki.**Method:** Fieldwork was carried out in a 2-year training course in agile development. This was validated in two phases (with and without PAL), which were subdivided into two stages: Training and Project.**Results:** The study demonstrates that, on the one hand, the learning process can be facilitated using PAL to transfer software process knowledge, and on the other hand, products were developed by junior software engineers with a greater degree of independence.**Conclusion:** PAL, as a knowledge repository, helps software engineers to learn about development processes and improves the use of agile processes.

© 2011 Elsevier B.V. All rights reserved.

1. Introduction

This paper focuses on the use of wikis such as Web 2.0 knowledge repositories to help software engineers access, query and reuse process assets in projects and, consequently, improve the ease of learning and use of software engineering best practices.

A software process is a coherent set of policies, organizational structures, technologies, procedures and artifacts that are required to design, develop, install and maintain software products [16,56]. Software engineers are not usually motivated to follow the process if they lack the supporting technology to create, store and search for the knowledge needed to develop their activities [13,47]. Moreover, increasing employee turnover rates and a more mobile workforce intensify the need for knowledge about processes and projects to be stored and reused [5,12]. Time and effort to describe a project process are reduced when a reusable process is instantiated instead of building the process from scratch [48].

Organizations need technological infrastructures to manage software process knowledge so that users can learn, use and adapt the process to carry out projects, by using repositories that foster collaboration. One such tool is a Process Asset Library (PAL). PAL is a knowledge repository used to store and make available all

the process assets that are useful to those who define, implement, and manage processes in the organization [11,57]. The stored process assets must be available for use in future projects and for defined software processes to be adapted for specific projects [6].

The objective of process assets is to help capture, understand and use the process, saving effort and thus improving productivity [23]. The assets generated in past projects can be stored at the end of the projects and reused in future projects. Reuse of these artifacts can save developers time and effort because the assets encapsulate the experience of the organization.

The use of a PAL is a required practice to achieve Level 3 of CMMI [11]. PAL facilitates standardization and process improvement in organizations and is a key enabler for achieving higher capability maturity [15].

Investments in designing and deploying processes are beneficial to small and large software intensive companies:

- For a small organization, PAL is a key infrastructure element that:
 - Reduces training time to introduce improvements in the software process.
 - Helps lead to a process-focused culture that provides a backbone to the discipline.
 - Provides a key element to support a reduction in the time needed to plan new projects, a typical area where small (as well as large) projects are challenged.

* Corresponding author. Tel.: +57 68741367, fax: +57 8879300.

E-mail addresses: jgarcia@inf.uc3m.es (J. García), amescua@inf.uc3m.es (A. Amescua), misanche@inf.uc3m.es (M.-I. Sánchez), lbermona@unal.edu.co (L. Bermón).

- Increases the capability to rapidly deploy and use processes to serve the needs of the marketplace.
- For a large organization, PAL provides one of the infrastructure elements required to support movement from one set of behavior to another by making public the “new rules” that the organization intends to live by. A well-designed and deployed PAL also reduces planning, implementation, and training time in large organizations, especially for processes that are only intermittently performed.

However, for PAL to be effective, it is necessary to develop domain-specific process assets that can be easily stored and updated [39]. Moreover, PAL is required to capture the diverse forms of knowledge gained from projects [23]. It is difficult to ensure that new processes are adopted in organizations because are conditioned by existing relationships between people and artifacts [21]. PAL must store useful and accessible artifacts to help create the work products of the projects. Also, it must provide learning capabilities to develop collective skills, and be an effective learning environment for training new employees [6].

The objective is for software engineers to have greater autonomy in learning; there is greater emphasis on active learning, with creation and participation playing key roles [52]. During training, software engineers may be involved in a few small software development projects that must be accomplished quickly. At the same time, they must obtain theoretical knowledge of software processes [27]. Therefore, tools are required to provide different support levels for learning and using processes [16].

This work is related to the works of several authors [4,51] who recommend knowledge management strategies and the use of wikis to create and maintain process information collaboratively and dynamically [34,42].

Although the number of software processes knowledge repositories is limited, in agile development it is even more limited, mainly to file repositories [10]. In this regard, the challenge was twofold: first, we defined the knowledge management mechanisms that allow PAL to facilitate learning and using the software process. Then, we applied the theoretical guidance defined for agile development processes repositories.

To meet these challenges we defined a PAL, based on Knowledge Management, implemented through a wiki in order to:

- Facilitate agile development processes learning by storing and reusing the process assets.
- Improve software engineers' accessibility to the processes learned.

To address the above objectives, this study proposes the use of Knowledge Management, which targets the capture, codification, and dissemination of knowledge across the organization to enhance value [1]. In software engineering, there has been much discussion about how to manage knowledge [7]. To facilitate the development of knowledge management systems, Web 2.0 offers a more interactive and collaborative framework of technologies, emphasizing peer social interaction and collective intelligence [38]. Knowledge management in PALs will increase skills to develop and store knowledge of organizational processes [14,25]. Web 2.0 includes the concepts of weblogs, wikis, really simple syndication (RSS) functionality, social tagging, mashups and user-defined content. As a vehicle for implementing the solution proposed, the concept of Wiki is used.

Wikis are among the 17 best learning channels evaluated in terms of cost (expenses a learner is likely to incur using the learning channel); learning efficiency (the amount of calendar time required to transmit a given amount of new knowledge to learners; learning effectiveness (the volume of information that the learning

channel can transmit to a learner) and currency of information (the average age of the information that is being transmitted). In the ranking of software learning channels, wikis are positioned in eighth place with the following values: cost (2), learning efficiency (9), learning effectiveness (16), and currency (3), where “1” is the top rank or score for each category [26]. In this ranking, only two channels out of the first seven support information and communications technologies (Web browsing and Webinars/e-learning). Unlike these two channels, wikis offer advantages in that they do not rely on search engines, knowledge can be formalized and the representation of knowledge can be maintained.

Wiki technologies are used as a collaborative web-based learning tool that provides opportunities for instantaneous revision and develops an efficient repository of new information [17]. Wikis support cooperative/collaborative learning and constructivist learning paradigms. Cooperative paradigms imply that wikis can serve as a knowledge platform for a community of practice where members of the community can share their knowledge with the group, put up interesting pieces of information, discuss issues, etc. Constructive paradigms imply that learners are engaged in interactions and explorations with learning materials and the wiki enables new ideas to integrate with previous knowledge.

Learners regularly use wikis as a source of information. Schaffert et al. [49] identify several tasks where wikis can be used: cognitive apprenticeship, learning within communities of practice, project-based learning, collaborative story writing, as well as interdisciplinary and intercultural learning. Parker and Chao [41] identify several benefits of the applications of wikis. Among these are higher levels of thought and information retention than those achieved when students work individually, enhanced peer interaction and group work, knowledge sharing and expertise among learners, and encouraging cooperation.

PAL can be implemented by a wiki to behave like a lightweight knowledge repository [44] in order to store, organize, distribute, reuse and preserve process assets; generate shared knowledge; increase the ability to rapidly deploy and use processes; help lead to a process-focused culture that provides a discipline for the organization and help processes learning and increase the quality of the products.

A wiki-based PAL to support agile software process learning and use was validated in a training course.

This paper is organized as follows. Section 2 presents works related to PAL. Section 3 describes the proposed solution. Section 4 describes the validation process. Section 5 analyzes and discusses the results. Finally, in Section 6, the authors present their conclusions.

2. Related work

On the Web, there are some specific repositories that implement corporate PALs. The Spawar System Center Pacific's PAL [18] and Goddard Space Flight Center of NASA's PAL [43] include artifacts such as guidelines, policies, standards, procedures and training material. These have a large amount of information that makes it difficult to understand the processes. They are organized by areas or categories but the assets are dispersed, so it is necessary to represent the evolution of assets and to include easy and flexible editing capabilities.

Calvo-Manzano et al. [9] developed a PAL oriented to small and medium enterprises. This PAL is based on Web 1.0 and its components are oriented to the Capability Maturity Model (CMMI) process areas, classifying them into practices, sub-practices and products. Our work differs in that it uses Web 2.0 technology that enables the upgrading of assets, based on a collaborative process, through editions and feedback from users and the ability to instan-

tiate and store assets used in software projects. These are all oriented to agile processes learning.

In the literature, wikis are used to support activities such as requirements and specifications, architectural models and test cases [45]. Software development teams have begun using wikis in processes, for example project management and tracking [32]; communication and collaboration, including brainstorming sessions [37]; organization of fragmented knowledge and exchange of information [20]; technical documentation of software [2]; requirements management and traceability [53]; architectural design [55]; and management of lessons learned [30].

There are few studies on the use of wikis as PALs. Wongboonsin and Limpiyakorn [59] developed a PAL using a wiki. The process assets used in their study are based on the CMMI model. Our study differs in the process structure, the types of artifacts used and the experiment conducted during software practices and projects.

In general, the related works lack functionalities to address some relevant aspects of PALs such as assistants to deliver information about processes and to support the interpretation and application context [19]; an integrated approach to codification (to systematize and store information that constitutes the knowledge of the company and to make this available to the professionals); personalization (to support the flow of information in a company by having a centralized store of information about knowledge sources) [7]; support for reuse of non-structured elements [22]; and monitoring and measurement tools.

The studies mentioned should include knowledge management capabilities to manage process assets (creation, acquisition, organization, distribution, use and preservation of knowledge) [4,58]. In activities such as knowledge acquisition and organization, these proposals store formal, structured and standardized knowledge according to a reference model (CMMI, IEEE 12207, etc.). On the other hand, informal knowledge about the process is generally not included. In terms of knowledge distribution or transfer, search and recovery of assets functionalities are provided, but with very simple formats. Regarding the use and reuse of knowledge, the tools do not include the ability to instantiate assets to be used in specific software projects. Finally, the evolution and preservation of knowledge should be done collaboratively in a PAL.

The main handicap of the works mentioned above is that the PALs implemented are no longer static repositories of information and must become useful and active knowledge repositories of software processes. Systems based on Web 2.0 such as wikis offer advantages for developing and implementing lightweight knowledge repositories. The processes of traditional knowledge manage-

ment such as creation, transfer and storage/retrieval are supported by wikis/wiki systems [50]. Wikis are easy-to-use knowledge sharing tools and can be oriented to helping development teams collaborate to carry out activities and manage software projects through a centralized repository of artifacts.

3. Solution proposed

The purpose of implementing PAL via a wiki is that this technology allows to implement a lightweight repository of knowledge that has impact on the knowledge management of software processes. Wikis are open, incremental and simple technologies. They address knowledge management issues such as usability, distributed access, diversity of knowledge, contended search, version control and content organization [54]. In software processes, wikis are among the software engineering best practices, offering advantages for increasing rapidly in usage and effectiveness. Wikis are now considered as new and emerging channels of learning in software engineering and are very appropriate for technical issues such as agile development, software quality and other topics where there are many practitioners with a need to share new data and experiences [26].

This paper seeks to apply knowledge management concepts to process assets through a PAL-Wiki to facilitate learning and use of software processes. The proposal offers an integrated approach to knowledge management strategies, based on codification and personalization of process-related artifacts, and allows users to share knowledge. It also incorporates knowledge management activities in order to acquire, organize, distribute, use, preserve, manage and measure the assets. The guidelines for building a knowledge management-based PAL include the definition of a structural model, and an incremental processes model. Moreover, an example of the proposed PAL is applied to agile processes.

3.1. Structural model

Assets stored in PAL-Wiki were identified and organized into a structure of elements. This structure is based on several software process models [24,28] and organizational memories in knowledge management systems [25,29,36]. An extract of the meta-model that represents this structure is presented as a UML (Unified Modeling Language) class diagram in Fig. 1. The most important ideas underlying the model can be summarized as follows.

PAL-Wiki consists of a set of *Process Elements*. The Process Elements can be standard processes, life cycle models, tailoring

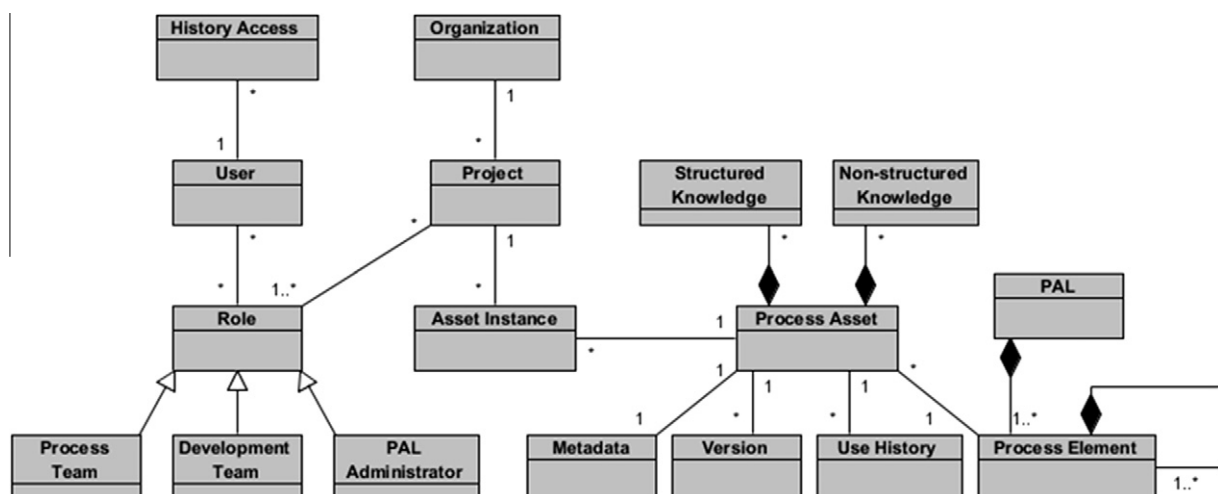


Fig. 1. Part of UML meta-model for the PAL-Wiki structure.

guidelines, process measurements or work environment standards. The Process Elements are associated with many *Process Assets* that can contain *Formal Content* or *Informal Content*. Formal Content refers to structured elements such as checklists and templates that implement codification strategies. Informal Content refers to unstructured elements such as lessons learned and discussions that implement personalization strategies. The Process Assets may have many *Versions* and can be instantiated and re-used (*Asset Instance*) in specific *Projects*. During the projects, there are three kinds of *Roles*: *Process team* produces, evaluates, edits and enhances knowledge; *Development team* consumes knowledge stored during the projects; and *PAL Administrators* manage users and contents. Reports on users access (*Access Historial*) and process assets are obtained from actions performed by *Users* in PAL-Wiki.

3.2. Processes model

The implementation of PAL-Wiki in an organization is a gradual process consisting of three kinds of functions based on integrative and interactive features of knowledge management systems [33,60]. The PAL-Wiki functions are shown in Fig. 2 and are briefly described below.

- **Core functions:** these functions implement basic knowledge management processes in PAL-Wiki.
 - **Knowledge acquisition:** This is the first function to be used in PAL-Wiki and allows implementing knowledge codification and personalization strategies. Process knowledge is obtained with sufficient detail to be transferred, stored, and used in an organization. This knowledge is acquired and codified through forms and templates for different process elements and process assets. It is a collaborative activity

performed by the Process Team. Knowledge acquisition allows domain-specific process assets that can be easily stored. The personalization strategy is implemented through a collection of lessons learned and best practices of the processes stored.

To perform this function a process guide is defined using forms and templates on roles, processes, activities and work items. Through different forms, the Process Team capture detailed knowledge of the components of each of these elements and this knowledge is stored in PAL-Wiki.

A role is defined in PAL-Wiki by using a form that specifies the name of the role and a description of their associated responsibilities. A team member who is involved in the development, use and maintenance of the software product should perform the processes, activities and tasks defined in PAL-Wiki.

The software processes are defined by means of a form that includes: overview, participating roles, entry criteria, when the processes are performing, activities that constitute the process, exit criteria, work items, work products, training materials (slides, videos), and examples. Some assets are available in different formats such as multimedia (presentations, images and video), text documents or spreadsheets. The activities definition form is similar to the process definition form; the difference is that the constituent elements of the processes are the activities, and the activities are broken down into tasks, which are atomic elements.

Work items are artifacts that are managed to complete the software development tasks. A work item is defined by means of a form that includes the following elements: overview, list of states and their transitions (including a state transition diagram) and a list of fields related to the work item. Name and description are defined for each field.

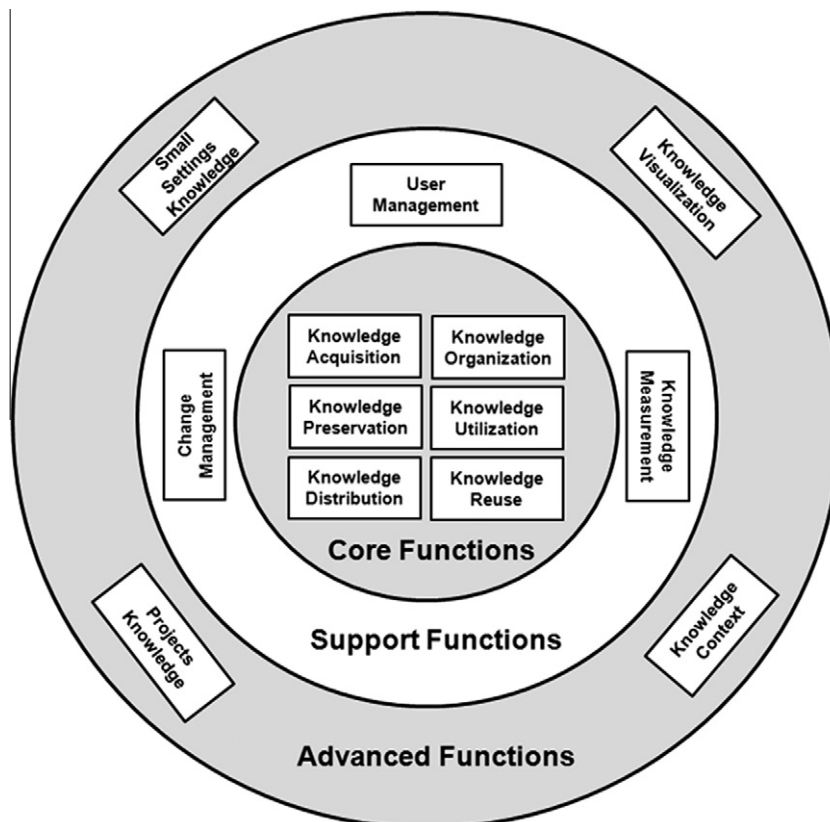


Fig. 2. PAL-Wiki functions.

Moreover, other forms can be created to adapt the process to the needs of each organization. These customized forms allow a flexible definition of the process and facilitate the acquisition of formal and structured knowledge. Forms also help to edit and improve the content and provide a common view of the contents of the defined process.

- *Knowledge organization*: The knowledge gathered by PAL-Wiki needs to be organized according to some established structure, hierarchy or classification. Once a knowledge item (process asset) is created, it must be located in the structure and be linked to other knowledge elements. The organization also requires the transformation of knowledge, depending on their use. The knowledge about software process must be rebuilt and organized to meet specific learning and training structures supporting multimedia formats and include examples of process assets applied to specific projects.

The contents of PAL-Wiki are structured as processes guidance. This structure is not static; the Process team can edit and modify it. The organization of knowledge is based on a typological structure based on categories, which are relationships between the different elements of PAL-Wiki.

The processes correspond to the highest-level items. They are broken down into activities, and activities into tasks. Both processes and activities have different associated process assets.

- *Knowledge distribution*: PAL-Wiki should provide facilities for effective selection of its elements through: a structure having multiple types of nodes with associative connections that allow to browse and query the contents, a search service based on keywords that belong to certain categories and the contents of stored assets. The contents of the assets emphasize procedural knowledge that involves nodes on: What actions take place, Who performed them, When they were done and How they can be implemented and linked with artifacts represented through documents, slides, video and files. During the execution of a project, learning or updating session, the Development team may search for any kind of knowledge about the process, which must be distributed. For this purpose, the development team must navigate through the major sections of PAL-Wiki starting from the homepage and, after prior validation, access the contents. The general structure for navigating in PAL-Wiki comprises the following elements: introduction to the software process, roles, work items, index of processes, and user help. PAL-Wiki provides mechanisms for searching, retrieving, and displaying the assets of corresponding processes. PAL-Wiki should allow text-based search by asset title and search by plain text in assets contents. Process assets found may be selected, accessed and used during execution process.
- *Knowledge utilization*: The fundamental aim of PAL-Wiki is that stored knowledge about the organization's software process actually be known by users and effectively implemented during project implementation. Process assets stored in PAL-Wiki may be templates with a standard format that must be filled in according to the features and needs of a project. The new files generated from these templates are called "instances" of the asset. Instances are files created in different formats (text documents, spreadsheets, tables, figures, etc.) and are generated by tools outside PAL-Wiki. The assets can be downloaded and instantiated (their contents adapted for application to specific projects). Instantiated process assets can be stored again in PAL-Wiki as examples of assets that will be accessed and reused in future projects. This function is a collaborative activity that facilitates knowledge sharing.

Instances of process assets published and stored in PAL-Wiki may be reused during the development of a specific project. During the projects, the development team can find instantiated assets and use them as guidelines and examples for a given activity by removing the application context for effective reuse. These examples will help novices use process assets to do a particular activity in a software process.

- *Knowledge reuse*: The assets can be downloaded and reused completely (without modification) in projects by users of the process. During project development, the assets can be found, instantiated and used as guidelines and examples for a given activity by removing the application context for effective reuse.

Novices can reuse the process assets and query examples included in PAL-Wiki in order to understand easily how to perform the activities in software projects.

- *Knowledge preservation*: It focuses on maintaining PAL-Wiki through a *collaborative* process to improve the content through annotations, comments and feedback from users. Thus, the personalization strategy of knowledge is implemented. PAL-Wiki allows users to add or update content based on their own knowledge, achieved through using statistics on the use of assets. Irrelevant or outdated knowledge should be removed from PAL-Wiki.

Knowledge is preserved through discussion of the contents of PAL-Wiki. The development team can make contributions to generate comments in discussion threads for each asset. Based on these discussions, the Process team can make editions and improvements on the related contents. The discussions outlined by the users generate informal knowledge about the process that allows creating lessons learned. These lessons learned would be reviewed later by the Process team, who will improve the contents of the assets, generating new versions of it.

The editing and enhancement of the assets contents is achieved as the assets are used in development projects, where the Development and Process Teams propose improvement specifications based on the comments in discussion threads about assets and statistical reports on the use of the processes defined in PAL-Wiki.

PAL-Wiki should ensure the integrity of knowledge over time. Unused, obsolete or outdated process assets are identified in the reports obtained from usage statistics of PAL-Wiki. Based on these reports, the Process team would decide on the removal of process assets. Controlled removal of assets will enable continuous process improvement based on feedback from users on the use of PAL-Wiki. Assets previously removed may be restored in any previously generated versions.

Inexperienced users can join and generate discussions about the contents in order to raise doubts and questions about how to perform a particular process, and expert users can answer these questions to help novices understand aspects of the process through the externalization of implicit knowledge. Expert users also edit and improve the content of the assets from their experience in previous projects.

- *Support functions*: these functions address activities that apply more generally to the processes in the organization.
 - *User management*: This function manages access policies for different groups of PAL-Wiki users. PAL-Wiki has no anonymous users; to access the system, users must be registered in any of the defined user groups. Depending on the group, users can view and edit content, create new process assets, participate in discussions and upload examples of activities for specific projects.

The PAL Administrator is responsible for managing PAL-Wiki users. PAL-Wiki provides special capabilities to: create process user groups, create users for a particular group process, modify the access rights of users and track the actions they take.

- *Change management:* PAL-Wiki records all changes that users make to the contents stored. This function allows to list the different versions of assets, compare different versions and, if necessary, reverse the changes made. Users can see an overview of a number of recent changes in the contents of PAL-Wiki or all of the changes over time.

Users can modify process assets. The changes in the contents of the assets must be registered and users can check the version history of stored assets. PAL-Wiki should generate a list, sorted by date, of different versions made for a specific asset showing an asset version identifier, name of user who made the changes and a link to view the actual content of the asset.

The contents of PAL-Wiki, which different users can edit, may contain errors (spelling, language, format, etc.). These errors also occur from contributions made in discussion pages. The PAL Administrator should regularly review the content and recent changes to detect and correct these errors.

- *Knowledge measurement:* To evaluate appropriate and effective knowledge management of the organization's software process, PAL-Wiki should provide capabilities to monitor the use of process assets stored in order to identify opportunities for improvement by introducing corrective changes. Through the collection of quantitative measures, reports are obtained on different actions that users perform on PAL-Wiki's assets (searches, queries, instantiations, etc.), and behavior patterns of users (accesses, contributions, editions, etc.)

To assess PAL-Wiki, reports should be generated on actions taken by users during their access and use of stored content. Among the statistics to assess user behavior are: number of PAL-Wiki users, number of users' access to PAL-Wiki per period of time, and contributions from users such as content creation and editing, participation in discussions, generation of lessons learned and publication of examples of process assets.

Moreover, reports should generate the access and use of process assets stored. PAL-Wiki should generate statistical reports with metrics such as: statistics on process assets (including pages, files and discussions), statistics on editions to process assets, statistics on users' visits to PAL-Wiki and most viewed and consulted pages.

- *Advanced functions:* these functions address processes that focus on the projects and deployment of processes.

- *Small settings knowledge:* Small companies live in an ever-changing world and they have little or no influence on the way their environment changes. Thus, anything that depends on a stable environment has little or no value for a small company. Small companies focus on being innovative and flexible instead of using large and bureaucratic processes. The repository provides assets with a summary and detailed contents for small and medium enterprises.
- *Projects knowledge:* Software organizations apply the assets stored in projects development. Through this function, PAL-Wiki stores knowledge about the projects undertaken by the organization, thus generating a repository of projects integrated with PAL-Wiki and creating tailored guidelines of processes from the types of projects stored.

PAL-Wiki has the ability to store knowledge about the projects developed by the organization. Users can find assets that have been used for those projects and transform them into work products that should also be stored.

Process assets found and used during the distribution processes and use of software process knowledge allow the creation of work products associated with a specific project that is being performed.

Instead of assets associated with the process as examples, these work products are associated with projects where they were generated; this helps to carry out activities in future projects.

PAL-Wiki should generate measurements on projects and work products stored. Some metrics that can help assess the current state of PAL-Wiki in relation to the projects are: number of projects implemented by the organization, number of work products generated by project, amount of work products generated by activity for each project, the relationship between work products and process assets stored, and effective utilization rate of the assets into the projects.

- *Knowledge visualization:* Because knowledge management presents barriers and problems regarding its development and operation, especially in codification and knowledge transfer, knowledge maps were developed as tools to visualize the sources, flows, restrictions and components of tacit and explicit knowledge. PAL-Wiki includes knowledge maps to coordinate, simplify, highlight and navigate the knowledge stored, and provides a visual framework to capture and organize knowledge areas.

Navigation of the structure of PAL-Wiki is an activity that is performed through a knowledge map instead of a hypertext-based visualization presented in knowledge distribution.

The knowledge map represents the software elements as a set of inter-connected nodes with a top-down representation. Usually, the process user is allowed to expand and retract the nodes and their associated sub-trees to adjust the degree of detail of knowledge displayed.

The PAL-Wiki pages are shown as inter-connected nodes. Nodes can be selected to display their content directly. When a knowledge map node is selected, it will be deployed on a small window with a summary of knowledge on the node. A page with the knowledge of the process element selected will be presented in type text formats in a nearby window.

- *Knowledge context:* For a more effective reuse of assets, structuring knowledge first requires decontextualization (elimination of application context), and then recontextualization when applied to a new situation or project.

3.3. An experience with agile processes

This PAL-Wiki meta-model, based on knowledge management, was instantiated for validation using elements of agile development process. PAL-Wiki included all core and support functions. The main functionalities of PAL-Wiki, oriented to agile development, are shown in Fig. 3.

The proposed PAL-Wiki model may be applied to any software development process whose process elements can be represented through a structured set of forms on process knowledge acquisition that make up a generic software processes guide. Therefore, it could have started with classical iterative processes, but starts instead with agile processes because they are well suited to the needs of the model defined and offer some advantages such as having a small set of assets that are integrated into the process being

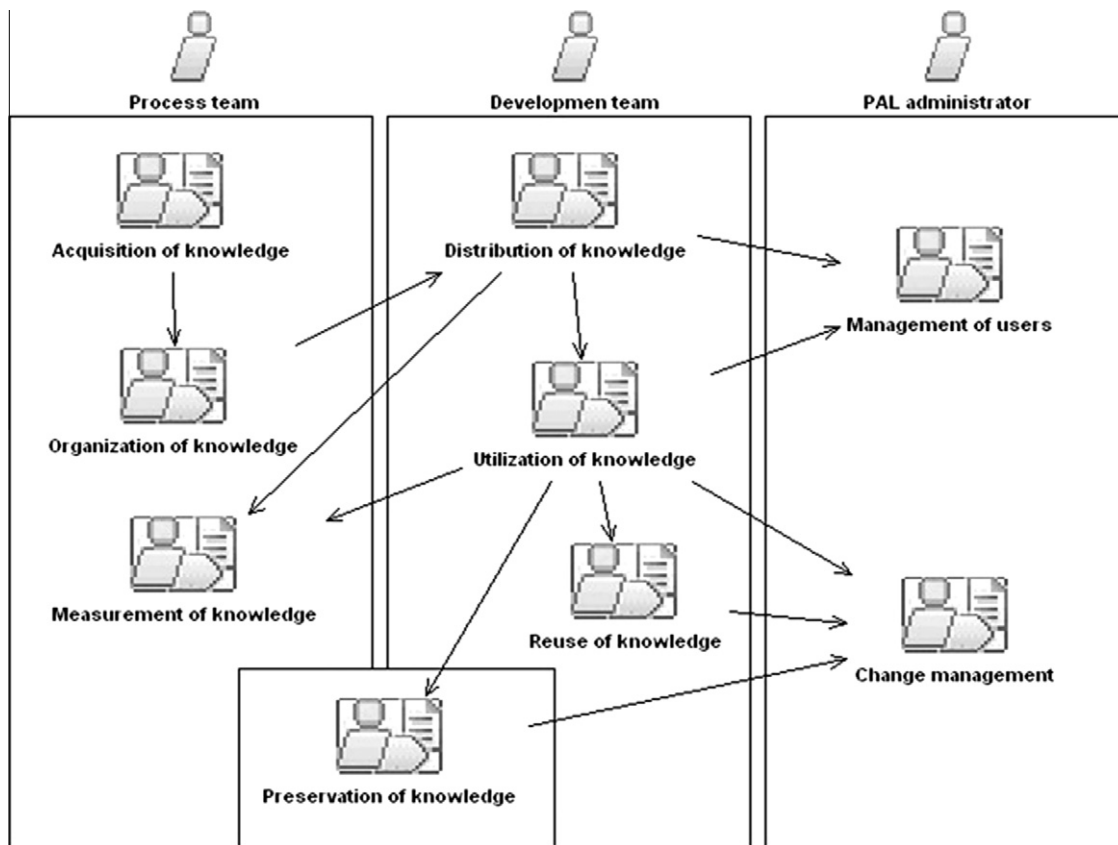


Fig. 3. Functionalities of PAL-Wiki based on knowledge management.

applicable to small projects, having less documentation and shorter iterative cycles, and fostering a culture of sharing tacit knowledge.

The meta-model in Fig. 1 was instantiated to generate the PAL-Wiki model oriented to agile development process. Agile processes consist of a minimum set of tasks that lead the project teams to build and deliver software products. Agile processes also require knowledge repositories that enable project teams to adapt lightweight but essential work products to perform the tasks. Although the number of software processes knowledge repositories is very limited, even more limited are those in agile development processes, which are generally applied repositories of isolated files without process integration [10]. The centralized nature of

PAL-Wiki is good to standardize process knowledge that, in agile methods, usually consists of a set of practices that must be balanced with disciplined processes [8].

Web 2.0 technologies that help software engineers access, view and reuse process assets and therefore support the learning and use of software engineering best practices can be applied through a PAL-Wiki on agile processes.

The main elements of this model are shown in Fig. 4. A brief description of the model elements is described below.

- **Processes:** Agile development processes such as user stories management, planning, development, refactoring, tracking, continuous integration, and defect management [3,31]. Each

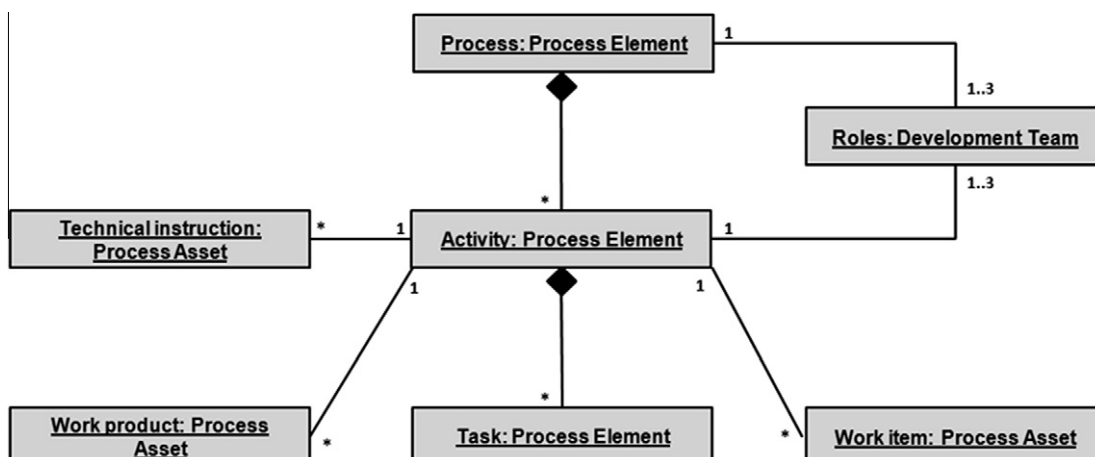


Fig. 4. PAL-Wiki model oriented to agile development processes.

process is defined by a general description, a list of roles that participate in the process, entry and exit criteria, a list of activities, work items, work products and technical instructions.

- **Activities:** Each process is made up of a set of activities. These activities are similar in structure to processes and are broken down into tasks.
- **Tasks:** Set of actions to be undertaken to successfully perform an activity. The tasks are bottom level elements that are not decomposed.
- **Roles:** Three roles are defined: The Team coordinator is responsible for co-coordinating, tracking and controlling the project; Software engineers develop the software product, and Customer/User provides the requirements of the software product to be developed.
- **Work items:** Records used to track and determine the progress of an activity or task. Work items defined are: defect, user story, quality of service requirement, resource, acceptance test, unit test, class, task, and automatic integration element.
- **Work products:** These are files, documents, specifications, binary files, parts and other tangible items that are necessary to complete the activities.
- **Technical instructions:** They provide information on carrying out activities and tasks. They may include additional resources such as sets of slides, videos, examples and links to discussion pages.

In total, PAL-Wiki has 7 processes, 31 activities, 9 work items, 6 work products, 3 roles, 22 sets of slides, 38 videos and 30 examples. Regarding implementation issues, PAL-Wiki was built by MediaWiki tool, which was installed on an Ubuntu Server with Apache Web Server, PHP programming language and MySQL database. It can be reached at <http://wikisel.sel.inf.uc3m.es/dhip/>, login: guest, password: guest01. The contents of PAL-Wiki are both in Spanish and English with links to translations at the top of each page.

4. Validation description

PAL-Wiki was validated in a training course on agile software development in which 66 junior software engineers participated. The validation was carried out in two phases, and each phase included two stages: Training and Project (both stages are described

in Sections 4.1 and 4.2 respectively). The timelines for each phase and stage are shown in Fig. 5.

Fig. 6 shows the main activities for the validation, the data gathering instruments and the data analysis mechanisms.

Different groups of engineers were involved in the two phases. The engineers' experience in software development processes was similar for each group, but none of the groups had any experience in agile methods and needed to learn, understand and use new software processes. Two trainers participated in the groups, one for theory and the other for practice.

For the training stage, practical sessions were developed. The amount and content of these sessions were similar in both groups. Learning activities were developed for different aspects of agile development in each session. The project stage consisted of modifying a multimedia file-management system by using the concepts of agile development methodologies learned in the training stage. The types of files that the system could manage were music, videos, photos and documents. The information the system manages is stored in XML files. Among the features to implement were: add files to the system, add folders, access files catalog, search for files by name or attributes, and status bar of the activities being undertaken.

In Phase 1, PAL-Wiki was not used and the trainers conducted a set of tutorials for the software engineers. However, the engineers used PAL-Wiki in Phase 2 and tutorials were not needed.

In Phases 1 and 2, the grades of the work-products created in the project stage were used as data collection instruments. The work products evaluated were: user stories files, acceptance tests, product plan, coding standard, design artifacts, refactoring, unit tests, collective ownership of code, continuous integration, tracking and iteration evaluation meeting. The instructors evaluated the quality of the work products. In addition, statistics on access to PAL-Wiki were collected to assess the number of users, access frequency and process assets consulted and used during the training and project stages in Phase 2. Surveys were also developed and implemented to enable students to evaluate the training and project stages in this phase.

Descriptive statistics, hypothesis testing and regression and correlation analysis were used to analyze and interpret data collected.

We formulated the following hypothesis: IF a PAL, based on knowledge management and implemented through a wiki, is

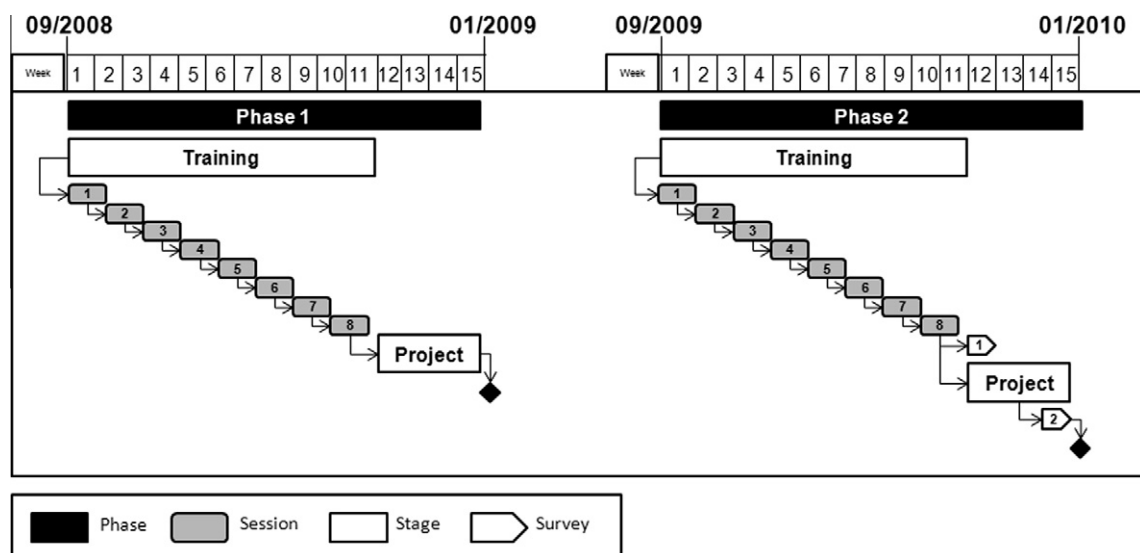


Fig. 5. Timelines for validation phases and stages.

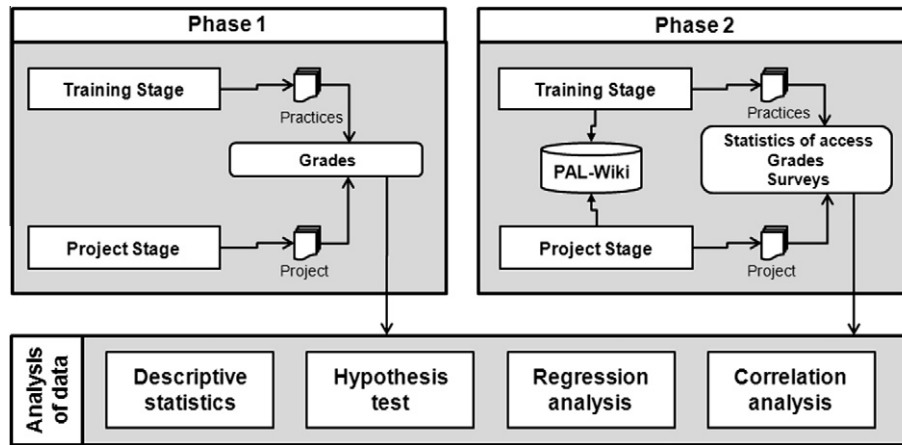


Fig. 6. Validation activities and instruments.

developed following the recommendations given in Section 3 THEN it

- Facilitates agile development processes learning by storing and reusing the process assets.
- Allows software engineers to use processes learned with a greater degree of independence.

The objectives of the training course in both groups were to:

- Acquire knowledge about new trends in agile software development.
- Apply techniques and concepts of agile and iterative software development.
- Apply practices and techniques to manage software development work teams
- Learn and use necessary tools to manage software development teams.

4.1. Phase 1: September 2008 to January 2009

Phase 1 was executed by 38 software engineers who did not use PAL-Wiki. This phase consisted of:

4.1.1. Training stage

This stage lasted 11 weeks. The guided exercises were divided into eight sessions: (a) User stories, (b) Acceptance tests, (c) Coding standard, (d) Simple design, (e) Refactoring, (f) Unit tests, (g) Collective ownership and continuous integration, (h) Tracking. The guided exercises were developed by teams consisting of two engineers and the grades of each group were collected.

4.1.2. Project stage

This stage started in week 12 and ended in week 15. The project involved the improvement of a small software application (multimedia file management) by using the concepts of agile development methodologies.

During this phase, the trainers conducted a set of individualized tutorials for the teams that expressed doubts about the application of the agile software development techniques.

The objective of the project was to develop a real software product because it contained a set of tasks, teamwork, environment, time pressure, and fairly realistic methods and techniques. At the end of the project stage, the grades corresponding to the quality of the work products were collected.

4.2. Phase 2: September 2009 to January 2010

Phase 2 was executed by 28 software engineers who used PAL-Wiki. Statistics on users' access and contents viewed and downloaded were collected. This phase consisted of:

4.2.1. Training stage

The duration of the training stage was 11 weeks. The guided exercises were the same eight sessions as in Phase 1. An introductory presentation was made on the access and use of PAL-Wiki at the beginning of the course. During this presentation, the following activities were undertaken: (a) Presentation of the overall hierarchy of contents based on process, activities, roles, work items, work products and technical instructions. (b) Query of process assets for specific activities during the development process. (c) Contributions from users via examples uploaded to the repository. (d) Creation of discussions about process elements through collaborative processes.

The engineers used PAL-Wiki for the guided exercises and practices. They became familiar with PAL-Wiki by exploring its contents. Once the 8 guided exercises were completed, an initial survey on the ease of use and benefits using PAL-Wiki to develop the practices was carried out.

The survey was based on the Technology Acceptance Model [40,46], evaluating the usability and usefulness of PAL-Wiki. The survey consisted of 21 items ranging from 1 to 5 on the Likert scale [35], 24 semantic differential items with 5 rankings and 4 open questions. The questions in this survey are listed in Appendix A. For this first survey, the items at the ease of use level were (questions A1–A11): visual presentation, navigation, structure, search, language, learning, artifacts, help, memorization, optional access, and ease of use. The usefulness items were (questions A12–A21): quality of products, time saved, easier work, ease of learning, completeness, usefulness of artifacts, visibility, communication of results, compatibility, and usefulness. The semantic differential items (questions A22–A25) assessed were: visual presentation of the elements, ease of access and understanding of the artifacts, easy to learn the concepts of agile development, and usefulness of the artifacts.

4.2.2. Project stage

The project stage started in week 12 and ended in week 15. During this phase, the engineers used PAL-Wiki and did not need any tutorials to clarify doubts on the application of agile software development techniques.

The characteristics of the final project were the same as the project in Phase 1. In both cases the grades of work products were collected to compare the results obtained with and without PAL-Wiki.

At the end of this stage, a second survey assessed PAL-Wiki's capabilities to provide information during the project. The questions for this survey are listed in [Appendix B](#). The second survey consisted of 18 items using the Likert scale (questions B1–B18), 10 dichotomous questions (questions B19–B28) and 13 semantic differential items with 5 rankings (questions B29–B30). The items assessed were: seeking knowledge, presentation of information, GUI, interaction, process control, artifacts, productivity, content, knowledge management, repository of projects, sharing knowledge, and capabilities for future works. The dichotomous questions were related to actions performed by users. The semantic differential items collected data on the quality of processes and artifacts.

5. Analysis of results and discussion

[Table 1](#) presents, for each validation objective, the part of the validation used to assess its compliance.

5.1. PAL-Wiki usability survey results discussion

Survey 1 assessed the usefulness and some usability aspects of PAL-Wiki during the training stage of Phase 2.

The results of the usability items analyzed are outlined in [Table 2](#). All the software engineers reported a simple and attractive visual presentation of PAL-Wiki. A majority of the software engineers reported the usability of PAL-Wiki in terms of structure and organization, language, easy to learn, easy to use artifacts, query help not being necessary, and in general, ease of use. However, the soft-

ware engineers had trouble finding contents and remembering the PAL-Wiki structure.

Item A-10 on voluntary access assessed the degree to which use of the PAL-Wiki is perceived as being voluntary or optional. The low response indicates that the user enters PAL-Wiki with the goal of resolving the practices that are mandatory to pass the training course. Therefore, access is not voluntary and PAL-Wiki needs to be tested in a real software development environment for a more accurate assessment of user satisfaction on voluntary use.

In general, the results of the items of the survey show that usability was perceived very favorably. The survey reflects a positive assessment of the knowledge repository (75.9%). The items related to the system usability were evaluated positively (71.11%).

In addition, 24 semantic differential items obtained the results shown in [Fig. 7](#). The visual presentation of the system ([Fig. 7a](#)) was rated favorably with an average of 3.84 and the “Processes” item was the highest rating because it is the repository core. Ease of access and understanding artifacts ([Fig. 7b](#)) obtained an average of 3.53, “Examples” was the best evaluated and “Discussions” got a low rating because they were seldom used. “Ease of learning” ([Fig. 7c](#)) was fairly even, with an average of 3.78. “Usefulness of the artifacts” ([Fig. 7d](#)) shows a similar high rating of access and understanding, with an average of 3.80.

Therefore, the results of the survey indicate that the knowledge repository developed (PAL-Wiki) was accepted by the users for its simple interface, ease of learning and ease of use of stored artifacts.

5.2. Objective 1: Facilitate agile development processes learning by storing and reusing the process assets

For this objective, surveys were conducted to assess what was learnt at the end of the training and project stages in Phase 2. A hypothesis test and a regression and correlation analysis were applied to determine the relationships between the use of PAL-Wiki and its benefits to perform tasks during Phase 2.

The survey data indicate a favorable response regarding the benefits and ease of use of PAL-Wiki. The response was positive for usefulness of stored artifacts, specifically for examples, videos and work product because they provide a practical approach to performing tasks. Processes such as “Refactoring”, “User stories management”, “Planning” and “Tracking”, which included many artifacts were positively valued. The theoretical material provided had a medium rating because they could not be downloaded and printed for viewing off-line. In general, ease of use was considered

Table 1
Validation objectives and validation elements used.

Objective	Validation elements used
Objective 1: Facilitate agile development processes learning by storing and reusing the process assets	• Training stage – Statistics on access to PAL-Wiki (users, accesses, access time, assets download) during the training stage. – Results of the survey.
Objective 2: Improve software engineers' access to the processes learned	• Project stage – Statistics on access to PAL-Wiki. – Level of product quality by process in each phase.

Table 2
Responses to usability survey.

Item	Agree (%)	Disagree (%)	No opinion (%)
A1. PAL-Wiki offers a simple and attractive visual presentation of the development process development	100	0	0
A2. It is easy to browse PAL-Wiki to find contents, materials and artifacts needed to perform the practices	60	40	0
A3. Structure and organization of development process contents in the PAL-Wiki are clear and help to perform specific tasks	86.7	13.3	0
A4. It is easy to search for information needed to perform a specific task using PAL-Wiki	53.3	40	6.7
A5. The wording of the contents of PAL-Wiki is clear and easy to read and understand	86.7	13.3	0
A6. PAL-Wiki is easy to learn to use and operate	93.3	6.7	0
A7. It is easy to use the artifacts stored in PAL-Wiki (work items, work products, videos, slides, examples) to develop projects	86.7	13.3	0
A8. It is often necessary to query the help available in PAL-Wiki to perform some actions	40	46.7	13.3
A9. It is easy to remember the content structure of PAL-Wiki	40	60	0
A10. I have accessed PAL-Wiki voluntarily	13.3	80	6.7
A11. In general, PAL-Wiki is easy to use	86.7	13.3	0

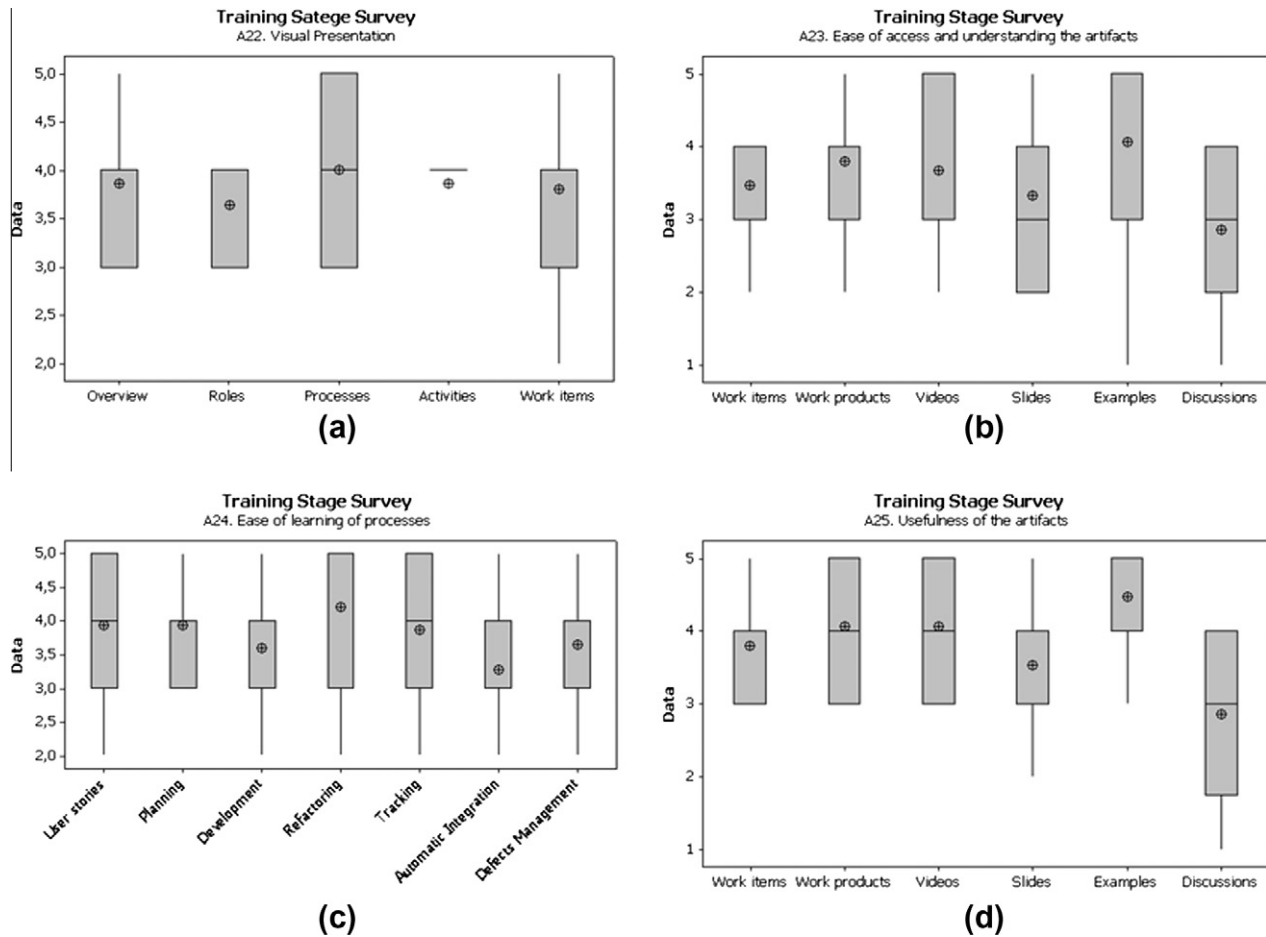


Fig. 7. Distribution of semantic differential items.

good (3.93 out of 5). The training survey indicates that PAL-Wiki is easy to learn and use.

Table 3 presents some statistics on access to the processes during the training stage of Phase 2 together with the learnability of each process from the survey.

A multiple regression was found ($R^2 = 0.997$) among the variables: users (U), accesses (A), time (T) and downloaded assets (D) that act as predictors of Learnability (L). The regression Eq. E1 obtained was:

$$L = 3.58 - 0.0157U + 0.000884A + 0.0887T - 0.00513D \quad (\text{E1})$$

Equation E1. Multiple regression about Learnability.

Eq. E1 shows that accesses, duration, and query and reuse of assets stored in PAL-Wiki influence software process learning:

- The variable, Users (U), has a negative relationship (coefficient = -0.0157) with respect to Learnability. This relationship

Table 3
Processes access statistics gathered during the training stage.

Processes	Users	Access	Medium time/user (min)	Assets downloaded	Learnability (1–5 scale)
User story	53	1075	11.23	149	3.933
Planning	63	1688	10.68	209	3.933
Development	117	2298	7.43	163	3.600
Refactoring	34	725	7.78	33	4.200
Integration	48	1281	8.77	175	3.867
Tracking	45	811	5.33	153	3.267

can be explained because the use of PAL-Wiki was continuous and sustained by users with access to computers where they studied. These indicators show that PAL-Wiki requires better outside access using different media to ensure a multi-point access and off-line use of the knowledge stored.

- The variable Access (A) represents the number of sessions that a user accessed PAL-Wiki during the training stage, considering that a user can have many accesses. This variable presents a positive relation (coefficient = $+0.000884$) to Learnability. The number of accesses indicates that users who often referred to the contents of PAL-Wiki acquire knowledge and learn about stored processes more easily.
- The variable Average access time per user (T) in minutes is the average time spent per user access when logging into PAL-Wiki during the training stage. This variable has a strong positive relationship (coefficient = $+0.0887$) with respect to Learnability, which indicates that the longer users spend on PAL-Wiki, the more content they will consult and the more software processes concepts they will learn about.
- The variable Downloaded Assets (D) represents the total number of process assets downloads stored in PAL-Wiki during the training stage. This variable has a negative relationship (coefficient = -0.00513) with respect to Learnability. This can be explained because the amount of assets to be downloaded is small and users need the access and understanding of the process guidance defined in PAL-Wiki to use the assets.

Eq. E1 expresses the participation of these four factors of PAL-Wiki's knowledge measure, each having a different relevance and

impact on the users' assessment of Learnability. The equation determines that PAL-Wiki helps users learn the processes defined. The knowledge repository implemented is useful for training staff in software development companies that need to update the training of their employees.

Facilitating software process learning is also demonstrated from a set of correlations found in the survey data in the training and project stages. Table 4 presents the variables considered in the correlations found and their corresponding correlation coefficient.

Correlation 1 shows that access and understanding of artifacts stored are related to their usefulness. Artifacts such as work items, work products, videos, slides and examples were evaluated positively according to access and understanding perceived by engineers during the training stage.

Correlations 2 and 3 show positive relationships between the training and project stages. Correlation 2 shows that access and understanding of artifacts stored during the training stage are related to the quality of the artifacts evaluated during the project stage. Correlation 3 shows that the ease of learning of the processes is related to the quality of process information during the project stage. Both correlations indicate the influence of the training stage as a learning and understanding activity to assess the quality of process information and the artifacts stored in PAL-Wiki during project development.

Correlation 4 indicates the association between the number of accesses to the PAL-Wiki processes and the quality of the artifacts of these processes evaluated by the students to do their tasks in the training stage. These results indicate that the greater the user's access and use of the knowledge stored in the PAL-Wiki, the higher the perceived quality of the artifacts. This shows that continuous use of the wiki is related to the capacity of stored artifacts to provide assistance in the execution of processes.

Moreover, the 24 semantic differential items of Phase 2 – Training stage related to visual presentation (question A22), ease of access and comprehension of the artifacts (question A23), ease of learning (question A24) and usefulness of the artifacts (questions A25) have equal distributions, using *t*-student tests with confidence interval of 95%.

Contributions made by users in PAL-Wiki are presented in Table 5. They refer to the work products developed and uploaded to PAL-

Wiki by the teams during the training and project stages. These values show that users perform collaborative activities during these stages.

During the course software engineers uploaded files containing work products from practices and projects executed. These work items could be downloaded and reused to implement new work-products created in subsequent activities. The total amount of data is shown in brackets. The data indicate continued involvement of the teams using PAL-Wiki. User stories, acceptance testing, planning, tracking and coding standards are documents that engineers uploaded. The design consisted of UML sequence diagrams and CRC cards. Collective code ownership, refactoring, unit testing and continuous integration were files with specific code for each work product.

To develop work products uploaded to PAL-Wiki, engineers conducted collaborative activities that facilitated intra-team but not inter-team learning due to fears of plagiarism of content when materials developed by one team are used by other teams. Teams worked on their own materials, reading, summarizing, using and discussing the process elements defined in PAL-Wiki. The work products generated are the result of intra-group activities.

5.3. Objective 2: Improve software engineers' accessibility to the processes learned

In Phase 2, software engineers worked more independently because they had learned about the process by searching knowledge and artifacts stored in PAL-Wiki; without the trainers' tutorials to clarify doubts on the application of agile software development techniques.

Access statistics to PAL-Wiki show continuous access and use of the contents stored in the training and project stages. This permanent access to PAL-Wiki to develop work products is related to the fact that users did not need extra mentoring and support from the instructors to perform their activities.

The quality of the products developed was analyzed in order to observe whether it was affected by the absence of instructor support. Engineers developed the projects seeking assets and information stored in PAL-Wiki. Instructors, despite the absence of mentoring, evaluated work products by measuring their quality

Table 4
Processes access statistics gathered during the training stage.

Correlation	Variable 1	Variable 2	R^2
1	Access and understanding of the artifacts stored per process from the survey in Phase 2 – training stage (question A23)	Usefulness of the artifacts stored per process from the survey in Phase 2 – training stage (question A25)	0.99
2	Access and understanding of the artifacts stored per process from the survey in Phase 2 – training stage (question A23)	Quality of artifacts per process from the survey in Phase 2 – project stage (question B30)	0.842
3	Ease of learning per process from the survey in Phase 2 – training stage (question A24)	Quality of process information from the survey in Phase 2 – project stage (question B29)	0.895
4	Number of accesses per process from wiki statistics	Quality of the artifacts per process from the survey in Phase 2 – project stage (question B30)	0.842

Table 5
Collaboration activities in the training and project stages.

Process	Work items uploaded – training stage	Work items uploaded – project stage	Process	Work items uploaded – training stage	Work items uploaded – project stage
User story	381 stories (30 files)	129 stories (12 files)	Collective ownership	225 files	167 files
Acceptance test	378 tests (15 files)	241 tests (7 files)	Unit test	225 tests (195 files)	214 tests (67 files)
Planning	938 tasks (15 files)	553 tasks (8 files)	Refactoring	120 files	No data
Coding standards	15 documents (15 files)	5 documents (5 files)	Tracking	45 files	24 files
Simple design	69 diagrams, cards (30 files)	169 diagrams, cards (15 files)	Continuous integration	15 elements (15 files)	8 elements (8 files)

Table 6

Quality criteria of work products.

Criteria	Value
<i>User story (15 points)</i>	
A list including all the goals of the iteration is provided	1
For each objective identified, its priority is established	3
For each objective identified, its size is established	3
For each objective identified, the effort in hours required is established	3
For each objective identified, detailed information of the user story is established	5
<i>Acceptance tests (10 points)</i>	
One or more acceptance tests are identified for each goal of the iteration,	2
The acceptance tests identified allow the detection of any potential errors	2
The purpose of each acceptance test was specified	2
Steps to be carried for each acceptance test were specified	4
<i>Adaptive planning of the iteration (13 points)</i>	
Maximum work load of practice teams was determined	3
The scope of the iteration was determined	1
The tasks necessary to achieve the objectives of the iteration were identified	1
Each one of the specified tasks was related to one goal of the iteration	1
The tasks included specific products that had to be achieved to certify their completion	3
Each task had a unique identifier and someone in charge	1
The effort (in hours) was estimated for each task	3
<i>Simple design (12 points)</i>	
A sequence diagram was made	3
Each of the classes identified was specified in the sequence diagrams	5
All existing classes in the source code were included in at least one sequence diagram	1
The public attributes and methods were the same as those found in the source code	3
<i>Refactoring (14 points)</i>	
All source code developed met the coding standards	2
There is evidence of the application of refactoring techniques in the source code	2
The source code provided implements the exception handling system proposed	4
The source code provided was modified to remove the launch of events for the Controller to communicate with View of the system	2
The source code provided implements the pattern “Singleton” to manage a single grid in the system	2
The source code provided implements the pattern “Singleton” to manage a single catalog in the system	2
<i>Unit Tests (30 points)</i>	
There was an integration test to check the function of the system Controller	4
Classes of unit tests referred to the different classes of the model containing tests that validated the correct implementation of the exceptions to the launching system	2
Classes of unit tests referred to the Grid class contained tests for the correct implementation of the pattern “Singleton” on the grid	1
Classes of unit tests referred to the Catalog class contained tests for the correct implementation of the pattern “Singleton” in the catalog	1
<i>There was a class that contained all unit tests of the public methods of the classes implemented for the goals:</i>	
Add a music file to the system	2
Add all music files contained in international conference on computer and information technology, Shanghai, China. IE EE Computer Society a folder to the system	2
Remove a music file from the system	2
Show songs from a category on the grid of music files	2
Open the folder that contains the file selected in the grid	1
Add the selected file in the grid to the favorites list	1
Remove the selected file in the grid from the favorites list	1
Manage playlists of music files	3
Show all tags of the song selected in the grid	1
Edit the tags of the song selected in the grid	1
Show information about the file containing the selected song in the grid	1
Manage images associated with the video selected in the grid	1
Manage the subtitles of the video selected in the grid	4
<i>Collective ownership of code (3 points)</i>	
The latest versions of the source code and unit tests are stored in the version control repository	1
The latest version of the auxiliary artifacts of the project is stored in the version control repository of the practices team	1
The latest version of all files included in the project is protected in the server	1
<i>Continuous Integration (8 points)</i>	
There is a file that allows automation of the integration tasks located in the version control server	1
<i>The configuration file of the automatic integration contains an objective that allows the generation of:</i>	
The DLL with the business logic implemented	1
The executable by linking the DLL from business logic and correct compilation of all the graphic elements of the user interface	1
The DLL to execute unit tests of the business logic	1
The results of the execution of unit tests of the business logic	1
There is a registry that maintains information on the results	1
The files obtained from a successful integration are stored in a folder accessible via web for all the team members (including instructors)	1
There is a historical record of the unit test runs of the project	1
<i>Tracking (16 points)</i>	
All tracking reports (on the progress of the work and tasks within the scope of the iteration, generated by the dates set as milestones) are provided	4
Each tracking report contains updated information about the implementation status of each of the acceptance tests identified	4
Each tracking report contains updated information about the implementation status of each of the tasks necessary to achieve the objectives of the project	4

Table 6 (continued)

Criteria	Value
Each tracking report contains preventive information of the iteration considered in the tracking method of accumulated value	4
Iteration wrap-up meeting (10 points)	
The agenda of the meeting was prepared and sent	3
The purpose of the meeting was identified and communicated	2
The results of the iteration were presented in the meeting	3
Irrelevant technical aspects were not discussed during a demonstration meeting	1
The clients were asked to give their opinions and recommendations for future improvement actions	1

levels. The instructors evaluated the quality level of each work product developed. A list of evaluation criteria for work products is presented in Table 6. Each part is assigned a value representing its relative importance. These criteria were also made known to the engineers so they could make a self-assessment of their work products.

Table 7 presents some descriptive statistics such as number of work products (N), the average score of the quality of work products (on a scale of 0–10) and standard deviation (SD) of the quality levels of work products evaluated.

To compare the quality levels of the two phases, we conducted a hypothesis test using t -student test having a confidence interval of 95%. This test analyzes the difference between the quality levels of products of the two phases to determine whether the difference is significantly important. The hypotheses defined were:

$H_0 : \mu_1 - \mu_2 = 0$ (Levels of product quality of both phases are equal)

$H_1 : \mu_1 - \mu_2 \neq 0$ (Levels of product quality of both phases are different)

In the results showed in Table 7, the p -values are greater than the significance level selected, so hypothesis H_0 cannot be rejected. Therefore, there is no difference in the quality of the products developed in the two phases. These results suggest that the quality of the products developed with or without PAL-Wiki is very similar but the degree of student autonomy increases because the instructors did not provide tutoring in Phase 2 and the engineers used PAL-Wiki to independently develop software products.

So we can affirm that although the use of PAL-Wiki does not improve the quality of the products developed, it encourages soft-

ware engineers to work more autonomously and by extension, the figure of the instructor is not required as much.

In PAL-Wiki, 28 user-accounts were created. The users used the assets during the training stage (11 weeks) and the project stage (4 weeks). During Phase 2, 217 pages were viewed for a total of 9028 times. The average time on site during the training stage was 8:43 min per user and 22:12 min per user during the project stage. The average number of pages per visit during the course was 12.45. The most popular type of pages was “Processes” (415 visits), because they were used as the entry point to the different assets. The pages related to user stories were the most downloaded: the work item “User Story” (309 visits) and the process “User stories management” (288 visits), as the first process in agile development.

Access was high in the first few weeks when the users did not know the PAL-Wiki structure. Then it varied according to practice deadlines. From week 12, the users began to work on the project stage and the final increase was because it was the last week to deliver the project. The trend in assets downloading was similar to the number of accesses, except in the final week because they had already been downloaded in previous practices.

The statistics collected show continued use of the process with more than one access per user each week. The results show that PAL-Wiki promoted a greater degree of independence for the software engineers during agile processes learning and resolved doubts about the description of the software products to be developed through storing and reusing the process assets.

6. Conclusions

This paper provides guidelines to build a PAL with a structure and functionalities based on Knowledge Management using a wiki system and an example of the proposed PAL applied to agile development processes.

The PAL-Wiki developed is featured to facilitate an effective learning environment to help users learn a process with a greater degree of independence from software engineers, provide knowledge sharing mechanisms and a knowledge repository with useful and accessible artifacts. The results show that software engineers found and reused the stored artifacts to learn this process. These results are consistent with the evidence that points to wikis as scalable ways for documenting processes and projects [32,45].

PAL-Wiki is a lightweight knowledge repository oriented to capturing, codifying, and disseminating knowledge about software processes. The core and support functions, based on knowledge management, defined in the guidelines were implemented during a course in agile software development. The process elements and assets were easily created, organized, updated and measured by the process team (trainers). The development teams (software engineers) searched, consulted and reused the process assets found to effectively create the work products during the training and project stages. Thus, PAL-Wiki provided support for learning and using processes.

Table 7

Levels of product quality.

Work product	Group	N	Mean	SD	T	P -value
User story	1	8	10	0	–	–
	2	6	9.9	0.3		
Acceptance test	1	8	9.7	0.7	1.52	0.179
	2	6	8.7	1.6		
Planning	1	8	8.4	1.9	2.13	0.06
	2	6	5.8	2.5		
Coding standards	1	8	5	5.4	0	1
	2	6	5	5.5		
Simple design	1	8	8	3.1	0.41	0.692
	2	6	7.5	1.7		
Continuous integration	1	8	7.8	1.1	–0.71	0.497
	2	6	8.3	1.5		
Unit test	1	8	4.1	1.4	0.46	0.658
	2	6	3.8	1.5		
Refactoring	1	8	9.4	1.8	1.8	0.110
	2	6	7.2	2.5		
Tracking	1	8	8.4	2.6	0.99	0.345
	2	6	7.1	2.5		
Iteration wrap-up meeting	1	8	7.7	2.4	–0.29	0.775
	2	6	8.2	2.8		

PAL-Wiki implements codification and personalization strategies. With respect to codification, PAL-Wiki helps to codify explicit knowledge through specific fields in forms for different process elements. For personalization strategy, PAL-Wiki creates spaces of tacit knowledge through discussion pages and collection of lesson learned, and provides an intuitive and simple knowledge maintenance cycle.

The results show that software engineers access, query and reuse different artifacts stored in order to learn the process. The simple and intuitive interface of the wiki allows rapid reference to the main sections. The knowledge repository developed contributes to the use of wikis as an infrastructure for implementing PALs; CMMI recommendations indicate what a PAL must include but not how to implement it. The repository has a positive impact on learning and using the agile process; it provides useful and easily accessible support material that contributes to achieving development processes tasks. Therefore, PAL-Wiki is an effective learning environment for training in new software processes.

Software engineers reported that PAL-Wiki is easy to learn, use and operate; the stored artifacts are useful for solving learning practices; it is a useful means of publishing process knowledge, and the reuse of assets improves the quality of the products. Given that PAL-Wiki was focused on agile development, future work will consist of the application of PAL-Wiki in other software development courses. Also, future work requires replication of results in real software development environments.

From this study, we can infer that a wiki system helps software development processes learning, it is appropriate for the training and project stages, it improves the use of agile processes and motivates engineers to explore and investigate concepts autonomously in order to internalize and institutionalize the knowledge about software process in organizations. Agile processes generally consist in a set of practices and we are proposing a set of assets and a guideline for its using in projects. Validation of the library was done for agile development but we want validate others development types in future works.

The agile process defined in PAL-Wiki relies on communication and collaboration to access and share knowledge. PAL-Wiki enables easy and effective sharing of both explicit and tacit knowledge; brings together knowledge from distributed individuals to form a knowledge repository; preserves knowledge through collaborative editing; and improves organizational knowledge dissemination. The validation results show that using few tightly integrated artifacts through process codification and personalization strategies facilitates agile process learning.

The regression equation and correlations obtained show that access and use of PAL-Wiki influence software processes learning. Learners can use a wiki as an e-learning tool to build a shared repository, which achieves a common understanding of the processes. PAL-Wiki supports the collaborative paradigm allowing examples of assets to be uploaded and then downloaded for use in projects, participation in discussions and collaborative editing of content, and the constructive paradigm engages learners in interaction with and exploration of learning materials.

Combining aspects of knowledge management and software process management helps organizations through the reuse of knowledge as the staff “can see what they know”: Knowledge can be disseminated and used to achieve the institutionalization of the process; it fosters a good process culture in the organization and, in turn, reduces training costs.

Acknowledgments

This work has been partially funded by the Spanish Ministry of Science and Technology through the TIN2009-10700 Project and

the Spanish Ministry of Industry through Projects PPT-430000-2008-54, PPT-430000-2009-0012 and PPT-430000-2009-0013.

Appendix A. Training phase survey

Items in Likert scale	
A1. PAL-Wiki offers a simple and attractive visual presentation of the development process	A2. It is easy to browse PAL-Wiki to find contents, materials and artifacts needed to perform the guided exercises
A3. Structure and organization of the development process contents in the PAL-Wiki are clear and help to perform specific tasks	A4. It is easy to search for information needed to perform a specific task using PAL-Wiki
A5. The wording of the contents of PAL-Wiki is clear and easy to read and understand	A6. PAL-Wiki is easy to learn to use and operate
A7. It is easy to use the artifacts stored in PAL-Wiki (work items, work products, videos, slides, examples) to develop projects	A8. It is often necessary to query the help available in PAL-Wiki to perform some actions
A9. It is easy to remember the content structure of PAL-Wiki	A10. I have accessed PAL-Wiki voluntarily
A11. In general, PAL-Wiki is easy to use	A12. The use of PAL-Wiki improves the quality of final products to be delivered during practice
A13. PAL-Wiki allows guided exercises to be performed in less time	A14. It is easier to perform the practices using PAL-Wiki
A15. PAL-Wiki facilitates concepts of agile development process learning	A16. The information provided by PAL-Wiki is relevant and sufficient for the practices
A17. PAL-Wiki offers useful and reusable artifacts to develop guided exercises (work products, videos, slides, examples)	A18. I've noticed that my classmates use the PAL-Wiki for the guided exercises
A19. It is easy to explain the benefits of using PAL-Wiki to others	A20. The contents of PAL-Wiki are compatible with statements of the guided exercises
A21. PAL-Wiki is useful for storing and publishing knowledge about the development process	
<i>Semantic differential items</i>	
A22. Evaluate the visual presentation of the elements of the PAL-Wiki (overview, roles, processes, activities, work items)	A23. Evaluate the ease of access to and understanding of PAL-Wiki artifacts to perform the guided exercises (work items, work products, videos, slides, examples, discussions)

Training phase survey (continued)

Items in Likert scale	
A24. Evaluate the ease of learning the concepts of agile development process (user stories management, planning, development, refactoring, oversight, automatic integration, defects management)	A25. Evaluate the usefulness of the artifacts of PAL-Wiki to develop guided exercises (work items, work products, videos, slides, examples, discussions)
<i>Open questions</i>	
A26. Which PAL-Wiki sections did you consult most?	A27. What problems did you have with PAL-Wiki?
A28. What benefits did you obtain from using PAL-Wiki?	A29. What improvements do you propose for PAL-Wiki?

Appendix B. Project phase survey

Semantic differential items	
B1. The attempts to find information in PAL-Wiki to complete the final project were often unsatisfactory	B2. The structure of PAL-Wiki, based on processes, activities, tasks, work items, and work products, was appropriate for the completion of the final project
B3. The graphical interface of PAL-Wiki was appropriate to understand the software process concepts	B4. Interaction with PAL-Wiki was clear and intuitive
B5. PAL-Wiki allowed to control the activities in order to implement the final project	B6. The artifacts stored in PAL-Wiki were enough to know how to apply agile development process in the final project
B7. The use of PAL-Wiki increased productivity during the development of the final project	B8. The contents of PAL-Wiki were consistent and accurate to execute the final project
B9. The files stored on PAL-Wiki could be easily reused for the final project	B10. PAL-Wiki promoted the creation of a community of users of the development process
B11. PAL-Wiki encouraged users to publish the products developed in the practices so that they could be shared with the community	B12. The SelCampus documents repository, associated with PAL-Wiki, was appropriate to send products to carry out the practices and the final project
B13. The discussions carried out within PAL-Wiki were useful for understanding the development process	B14. The contributions of users to PAL-Wiki (through page edits and samples uploaded) helped to understand the development process
B15. PAL-Wiki was appropriate to raise awareness of development processes	B16. PAL-Wiki provided mechanisms for sharing knowledge about software development processes

Project phase survey (continued)

Semantic differential items	
B17. The PAL-Wiki provided a suitable environment for software development processes learning	B18. I recommend using wiki technology in future software processes courses
<i>Dichotomous questions</i>	
Which actions did you perform in PAL-Wiki during the course?	
B19. Watch slides	B20. Watch videos
B21. Download samples	B22. Download work products
B23. Use the search engine	B24. Read comments in discussions
B25. Participate in discussions	B26. Upload samples
B27. Add content to an existing page	B28. Make small corrections to a page
<i>Semantic differential items</i>	
B29. Assess the quality of the information in the following sections of PAL-Wiki to complete the final project (user stories management, planning, development, refactoring, oversight, automatic integration, management of defects)	B30. Assess the quality of the information in PAL-Wiki elements to complete the final project (work items, work products, videos, slides, examples, discussions)

References

- [1] R. Adler, Knowledge engines for critical decision support, in: Knowledge Management Strategies: A Handbook of Applied Technologies. Chapter V, Information Science Reference, 2008, pp. 143–169.
- [2] A. Aguiar, G. David, WikiWiki weaving heterogeneous software artifact, in: Proc. of the 2005 International Symposium on Wikis, San Diego, CA, 2005, pp. 67–74.
- [3] M. Alshayeb, W. Li, An empirical study of relationships among extreme programming engineering activities, Information and Software Technology 48 (11) (2006) 1068–1072.
- [4] A. Aurum, F. Daneshgar, J. Ward, Investigating knowledge management practices in software development organisations – an Australian experience, Information and Software Technology 50 (6) (2008) 511–533.
- [5] R. Baskerville, A. Dulipovici, The theoretical foundations of knowledge management, Knowledge Management Research & Practice 4 (2) (2006) 83–105.
- [6] S. Bayona, J.A. Calvo, G. Cuevas, T. San Feliu, A. Sánchez, Process deployment in a multi-site CMMI level 3 organization: a case study, Computer and Information Science (2008) 147–156.
- [7] F. Bjornson, T. Dingsoyr, Knowledge management in software engineering: a systematic review of studied concepts, findings and research methods used, Information and Software Technology 50 (11) (2008) 1055–1068.
- [8] B. Boehm, R. Turner, Rebalancing your organization's agility and discipline, in: Proceedings of XP/Agile Universe 2003, New Orleans, US, Springer, 2003, pp. 1–8.
- [9] J.A. Calvo-Manzano, G. Cuevas, T. San Feliu, A. Serrano, A process asset library to support software process improvement in small settings, Software Process Improvement, Communications in Computer and Information Science 16 (25) (2008) 25–35.
- [10] T. Chau, F. Maurer, A case study of wiki-based experience repository at a medium-sized software company, in: Proc. of the 3rd International Conference on Knowledge Capture, 2005, pp. 185–186.
- [11] CMU/SEI-2006-TR-008, CMMI for Development, Version 1.2, Software Engineering Institute, 2006.
- [12] J. Davies, R. Studer, Y. Sure, P.W. Warren, Next generation knowledge management, BT Technology Journal 23 (3) (2005) 175–190.
- [13] J.C. Derniame, F. Oquendo, Key issues and new challenges in software process technology, SPT, Software Process Technology, Novatica, Spain 5 (5) (2004) 15–20.
- [14] J.S. Edwards, D. Shaw, P.M. Collier, Knowledge management systems: finding a way with technology, Journal of Knowledge Management 9 (1) (2005) 113–125.

- [15] S. Fogle, et al., The Benchmarking Process: One Team's Experience, Software Productivity Consortium, IEEE Software September/October, 2001.
- [16] A. Fuggetta, L. Sfondini, Software engineering methods and technologies, in: R.H. Thayer, M. Dorfman (Eds.), *Software Engineering – The Supporting Process*, third ed., vol. 2, IEEE Computer Society Press, 2005, pp. 313–320.
- [17] V.Z. Gavvani, V.V. Mohan, Application of web 2.0 tools in medical librarianship to support medicine 2.0, *Webology* 5 (1) (2008).
- [18] B. Groarke, Running an SEPG – How SSC San Diego Operates its SEPG, SPEG 2006, Software Engineering Institute, 2006.
- [19] J. Grudin, Enterprise knowledge management and emerging technologies, in: *Proc. of the 39th Annual Hawaii International Conference on System Sciences, HICSS*, IEEE Computer Society, Washington, DC, vol. 03, 57.1., 2006.
- [20] H. Hasan, J.A. Meloche, C.C. Pfaff, D. Willis, Beyond ubiquity: co-creating corporate knowledge with a wiki, in: *International Conference on Mobile Ubiquitous Computing, Systems, Services and Technologies, UBIComm '07*, 2007, pp. 35–40.
- [21] W. Humphrey, TSP: Leading a Development Team, Addison-Wesley Publishing Company Inc., Reading, 2005.
- [22] A. Hyung-Jun, L. Hong-Joo, C. Kyehyun, P. Sung-Joo, Utilizing knowledge context in virtual collaborative work, *Decision Support Systems* 39 (4) (2005) 563–582.
- [23] P. Jalote, *Software Project Management in Practice*, Addison-Wesley, 2002.
- [24] D. Jeffery, F. Kurniawati, The use and effects of an electronic process guide and experience repository: a longitudinal study, *Information and Software Technology* 48 (7) (2005) 566–577.
- [25] M. Jenne, L. Olfman, *Organizational Memory, Handbook on Knowledge Management*, vol. 1, Springer, Berlin, 2003, pp. 207–234 (Chapter 11).
- [26] C. Jones, *Software Engineering Best Practices: Lessons from Successful Projects in the Top Companies*, McGraw-Hill, 2009.
- [27] G. Kelečić, Ž. Car, Teaching software process: an experience in implementing RUP in a student project, in: *Proc. of the 8th International Conference on Telecommunications ConTEL 2005/Gledac, Gordan*, 2005, pp. 479–484.
- [28] M. Kellner, U. Becker-Kornstaedt, W. Riddle, J. Tomal, M. Verlage, *Process guides: effective guidance for process participants*, in: *Proceedings of 5th International Conference on the Software Process: Computer Supported Organizational Work*, 1998, pp. 11–25.
- [29] R. Klamma, S. Schlaphoff, Rapid knowledge deployment in an organizational-memory-based workflow environment, in: *8th European Conference on Information Systems (ECIS 2000)*, Vienna, Austria, 2000, pp. 364–371.
- [30] A. Korthaus, M. Aleksy, S. Henke, A distributed knowledge management infrastructure based on a topic map grid, *International Journal of High Performance Computing and Networking (IJHPCN)* 6 (1) (2008) 66–80.
- [31] L. Layman, L. Williams, D. Damian, H. Bures, Essential communication practices for extreme programming in a global software development team, *Information and Software Technology* 48 (9) (2006) 781–794.
- [32] P. Louridas, Using wikis in software development, *IEEE Software* 23 (2) (2006) 88–91.
- [33] R. Maier, *Knowledge Management Systems: Information and Communication Technologies for Knowledge Management*, Springer, Berlin, 2007.
- [34] A. Majchrzak, C. Wagner, D. Yates, *Corporate Wiki Users: Results of a Survey, WikiSym'06*, Odense, Denmark, 2006, pp. 99–104.
- [35] G. Melnik, F. Maurer, A cross-program investigation of students' perceptions of agile methods, in: *Proc. of the 27th International Conference on Software Engineering (ICSE)*, ACM Press, 2005, pp. 481–488.
- [36] G. Mentzas, D. Apostolou, R. Young, A. Abecker, Knowledge networking: a holistic solution for leveraging corporate knowledge, *Journal of Knowledge Management* 5 (1) (2001) 94–106.
- [37] N. Mullick, M. Bass, Z. Houda, P. Paulish, M. Cataldo, Siemens global studio project: experiences adopting an integrated GSD infrastructure, in: *Proc. of ICGSE'06*, 2006, pp. 203–212.
- [38] S. Murugesan, Understanding web 2.0, IEEE Computer Society, IT Professional 9 (4) (2007) 34–41.
- [39] B.A. Nejme, W.E. Riddle, The PERFECT approach to experience-based process evolution, in: M. Zelkowitz (Ed.), *Advances in Computers*, Academic Press, 2006.
- [40] S.Y. Park, An analysis of the technology acceptance model in understanding university students' behavioral intention to use e-learning, *Educational Technology & Society* 12 (3) (2009) 150–162.
- [41] K.R. Parker, J.T. Chao, Wiki as teaching tool, *Interdisciplinary Journal of Knowledge and Learning Objects* (3) (2007).
- [42] N.M. Radziwill, A.L. Shelton, TWiki as a platform for collaborative software development management, *Advanced software, control, and communication systems for astronomy*, in: *Proc. of the SPIE, Glasgow, UK*, vol. 5496, 2004, pp. 609–617.
- [43] D. Raffo, W. Wakeland, Moving Up the CMMI Capability and Maturity Levels Using Simulation, Technical Report CMU/SEI-2008-TR-002, Software Engineering Institute, 2008.
- [44] E. Ras, S. Weber, *Software Organization Platform: Integrating Organizational and Individual Learning, Wikis for Software Engineering Wikis4SE ICSE*, 2009, pp. 56–66.
- [45] T. Riechert, T. Berger, Leveraging Semantic Data Wikis for Distributed Requirements Elicitation, *Wikis for Software Engineering, Wikis4SE ICSE*, 2009, pp. 7–13.
- [46] J.C. Roca, C.M. Chiu, F.J. Martinez, Understanding e-learning continuance intention: an extension of the technology acceptance model, *International Journal of Human-Computer Studies* 64 (8) (2006) 683–696.
- [47] F. Ruiz-González, G. Canfora, Software process technology: improving software project management and product software process technology, *The European Journal for the Informatics Professional* (5) (2004) 3–5.
- [48] X. Ru-Zhi, H. Tao, D. Chu, Y. Xue, L. Qian, Reuse-oriented process component representation and retrieval, in: *International Conference on Computer and Information Technology, Shanghai, China. IEEE Computer Society*, 2005, pp. 911–915.
- [49] S. Schaffert, D. Bischof, T. Bürger, A. Gruber, W. Hilzensauer, S. Schaffert, Learning with Semantic Wikis, in: *First Workshop SemWiki2006 – From Wiki to Semantics, co-located with the 3rd Annual European Semantic Web Conference (ESWC)*, Budva, Montenegro, 11th–14th June, 2006.
- [50] A. Scherp, F. Schwagereit, N. Ireson, Web 2.0 and traditional knowledge management processes, *KSM2009 1st workshop on knowledge services & Mashups*, in: *5th Conference Professional Knowledge Management, Solothurn, Switzerland*, 2009, pp. 25–27.
- [51] I. Scorta, A knowledge management practice investigation in Romanian software development organizations, *WSEAS Transactions on Computers* 8 (3) (2009) 459–468.
- [52] S. Sharp, N. Baddoo, S. Beecham, T. Hall, H. Robinson, Models of motivation in software engineering, *Information and Software Technology* 51 (2009) 219–233.
- [53] C. Silveira, J.P. Faria, A. Aguiar, R. Vidal, Wiki based Requirements Documentation of Generic Software Products, *AWRE'05*, Melbourne, Australia, 2005, pp. 42–51.
- [54] C. Soles, N. Ali, M.A. Babar, A Spatial Hypertext Wiki for Architectural Knowledge Management, *Wikis for Software Engineering, Wikis4SE ICSE*, 2009.
- [55] C. Solís, N. Ali, M.A. Babar, A Spatial Hypertext Wiki for Architectural Knowledge Management, *Wikis for Software Engineering, Wikis4SE ICSE*, 2009, pp. 36–46.
- [56] SWEBOOK, IEEE Computer Society Professional Practices Committee, Guide to the Software Engineering Body of Knowledge – 2004 Version, IEEE Computer Society, 2004.
- [57] R. Turner, Toward Agile systems engineering processes, *Crosstalk, The Journal of Defense Software Engineering* (2007) 11–15.
- [58] J. Ward, A. Aurum, Knowledge management in software engineering – describing the process, in: *Proc. of Australian Software Engineering Conference, ASWEC'04*, Melbourne, Australia, 2004.
- [59] J. Wongboonsin, Y. Limpiyakorn, Wikipedia customization for organization's process asset management, in: *International Conference on Advanced Computer Theory and Engineering*, 2008, pp. 467–471.
- [60] M.H. Zack, Managing codified knowledge, *Sloan Management Review* 40 (4) (1999) 45–58.